

Qualified Health

V1 vs V2 Comparison Report

Candidate Identification & Coordinator Dashboard

Generated 2026-09-26

Synthetic / demo data only - no real patient information is used anywhere in this system.

1. Scope

V1 is the initial Stage B build: FHIR-shaped ingestion, record linkage, eligibility engine, evidence extraction (LLM stubbed behind an ANTHROPIC_API_KEY presence check, no live key ever set), candidate API, and a minimal single-column frontend (candidate list + approve/reject). Deployed locally via Docker Compose. This is the version the user labeled "V0" after initial verification.

V2 is everything in V1, plus: a Groq-backed coordinator chatbot, two load-testing mechanisms, runtime (in-memory) Groq-key configuration from the UI, a bug fix for ungraceful LLM-error handling, and a full visual redesign of the dashboard.

2. Feature comparison

Area	V1	V2
LLM provider	Anthropic (placeholder only, never live)	Groq openai/gpt-oss-120b, live-capable
Evidence extraction	Structured claims + stub placeholder for notes	Structured claims + Groq-verified note statements, or a clear stub/error message
Chatbot	Not present	POST /chat - grounded Q&A, cites patient_key
Load testing	Not present	k6 script + built-in self-test (POST /loadtest/chat)
Runtime key config	Not present (env var + restart only)	POST /config/groq-key - in-memory swap, no restart, never persisted/logged
LLM error handling	N/A - no live LLM calls attempted	Every Groq call wrapped in try/except, degrades instead of 500
Frontend layout	Single column: header, list, approve/reject	Two-column dashboard: top bar, sticky sidebar, chat + settings panels
Frontend styling	Plain borders, flat buttons	CSS custom-property design system, hover elevation, button variants
API surface	/health /ingest /screen /candidates /finalize	Same, plus /chat /config/groq-key /loadtest/chat
Governance docs	Scoped to ingestion/eligibility/candidate-api + dashboard	Ownership extended to chat-api, key config, load-test, chat/settings panels

3. New API endpoints in V2

Endpoint	Purpose	Auth
POST /chat	Coordinator Q&A grounded in candidate data	X-API-Key
POST /config/groq-key	Runtime, in-memory Groq key/model swap	X-API-Key
POST /loadtest/chat	Built-in concurrent self-test against /chat	X-API-Key

4. Load-testing results (V2 only)

Run	Load	Result
k6 (chat_load_test.js)	Ramp 5->20 VUs, ~70s, 927 requests	0% failure, p(95)=18.42ms - both thresholds passed
Built-in self-test	5 workers x 3 requests = 15 requests	0% error, avg 20.98ms, p95 40.72ms
Built-in self-test (defaults)	50 requests	0% error rate

Note: without a live GROQ_API_KEY, all runs measured the stub-answer code path's latency, not real model latency. A live key would show materially higher (network-bound) latency.

5. Bug found and fixed during V2 hardening

Setting an invalid/expired Groq key via /config/groq-key and then calling /chat originally crashed with an unhandled groq.AuthenticationError, surfacing as a bare 500 Internal Server Error. Fixed by wrapping every Groq call (chatbot.answer, evidence_extraction._note_based_claim) in try/except, returning a clearly-marked degraded message instead - consistent with the graceful-degradation philosophy already used for the "no key set" case in V1.

6. What stayed the same

- Synthetic/dummy FHIR-shaped patient data (2 interventions, ~8 patients) - no real source-system access.
- Docker Compose topology: postgres (pgvector) -> backend -> frontend.
- X-API-Key / hmac.compare_digest auth on every write endpoint.
- The human-gate finalize flow (POST /candidates/{patient_key}/finalize) and its audit log.
- Scope boundary: this case study remains fully independent of other day5/ folders.

7. Known limitations / risks

- All data is synthetic; nothing here has been validated against real clinical records.
- The Groq key set via the UI is runtime-only - lost on backend restart by design; .env is the only way to set a key that survives a restart.
- No observability stack (traces/metrics) yet - Planned, not started.
- Load testing covers /chat only, not /screen /candidates /finalize - a deliberate scope choice.

8. Planned next (from detailed-plan.md)

Reusable plugin packaging, MCP server registration, OpenTelemetry + SigNoz observability, k6 result dashboard visualization, Obsidian knowledge vault, graphify knowledge graph, a prompt-engineering pass, and a first-user demo.